

엘브네아 — 개인 기여 문서

시스템 구현 · Unity C# 스크립팅

최지환 · 팀 「화석 피라미드」 3인 · 졸업전시 출품 (2025.10)

0. 이 문서를 따로 쓴 이유

팀 「엘브네아」 소개 포트폴리오는 28쪽 중 24쪽이 그래픽 디자인 산출물입니다. 목차가 01 Summary / 02 Graphic Design 두 축이고, 내용은 배경 원화 · 캐릭터 모델링 · 텍스처 · 지형 제작 · 굿즈 디자인으로 채워져 있습니다.

그 문서에는 게임이 실제로 동작하게 만든 작업이 설명되어 있지 않습니다. 타이틀 화면도, 전투도, 퀘스트 진행도 결과 화면으로만 등장합니다. 이 문서는 그 공백 — 제가 맡은 시스템 구현 — 만 따로 정리한 것입니다.

1. 프로젝트 개요

항목	내용
게임명	엘브네아 — 잊힌 낙원을 찾아서
장르 / 플랫폼	액션 어드벤처 · PC (Unity)
플레이 방식	싱글 플레이 · 메인 퀘스트 기반 진행
팀 구성	3인 (기획 · 아트 · 시스템 구현 = 본인)
담당	전투 · AI · UI 관리 시스템 C# 스크립팅, 대화 시스템 구축, 오디오 · 이펙트 연출, 빌드 안정화
산출	졸업전시 출품 빌드 (2025.10)

세계관 — 빛의 신 '엘리에르'가 봉인되고 어둠의 화신 '카르멜'에 의해 세계가 감정과 기억을 잃은 침묵에 빠진 뒤, 감응 능력을 지닌 유일한 존재가 탑을 정화하며 정령들과 감정을 공명해 나갑니다.

2. 3인 팀에서 내가 맡은 자리 — 그리고 하나의 기준

기획과 아트가 각각 한 명씩 있는 3인 팀이었습니다. 저는 그 둘이 만든 것을 실제로 돌아가게 만드는 쪽 전부를 맡았습니다. 구현 담당이 한 명뿐이라는 것은, 무엇을 직접 만들고 무엇을 가져다 쓸지 고르는 일까지 제 몫이었다는 뜻입니다.

그 판단에 쓴 기준은 하나였습니다.

이미 잘 만들어진 것은 가져오고, 이 게임에만 해당하는 것을 직접 만든다.

이 문서에 나오는 세 번의 선택 — 길찾기를 **NavMesh** 에, 상태 관리를 **Animator** 에, 대화를 **Pixel Crushers** 에 맡긴 것 — 은 전부 같은 기준에서 나왔습니다. 그리고 그렇게 아낀 시간은 엔진이 대신 해줄 수 없는 쪽, 즉 이 게임의 판단과 화면 흐름에 넣었습니다.

영역	한 일
UI 관리	게임의 모든 화면 전환과 표시 상태를 관리
전투	공격 · 피격 · 사망 처리, 스킬 4종, 무기 콜라이더 기반 명중 판정
AI	NavMesh 이동 + Animator 상태 머신 + 커스텀 판단 스크립트
대화 · 퀘스트	Pixel Crushers 를 개조해 퀘스트 · 아이템 · 사운드와 연결
연출	오디오 · 이펙트 배치, 파티클 시스템 최적화
로컬라이제이션	유니티짱(Unity-chan) 보이스 적용 및 크레딧 표기
빌드	디버깅 및 전시 출품 빌드 안정화

3. UI 관리 시스템 — 가장 많은 시간을 쓴 곳

게임에 등장하는 모든 화면을 한 곳에서 관리하도록 만들었습니다. 팀 문서 **ux/ui** 적용 페이지에 실린 화면들이 전부 이 시스템 위에서 동작합니다.

화면	구성
타이틀	Start · Continue · Option · Quit
로딩	썸 전환 중 표시
인게임 HUD	프로필 초상화 + 체력 바 · 재화 카운터 · 스킬 슬롯 4칸
대화창	NPC 이름 + 대사 영역 (예: 레리엔)
퀘스트	상단 배너 Quest No.1 : 주변을 둘러보자! · 우측 목표 리스트 · 상호작용 프롬프트 E 키를 누르세요
아이템 획득	아이콘 + 이름 + 설명 팝업
게임오버	나가기 · 다시하기

이 작업이 까다로웠던 이유

화면 하나하나가 어렵지 않습니다. 문제는 이것들이 동시에 돌 수 있다는 것입니다. 대화 중에 퀘스트가 완료되고, 그 순간 아이템을 획득하고, 뒤에서 몬스터가 때리면 네 가지가 한 화면에 겹칩니다. 화면을 각자 자기 상태만 챙기게 두면 이 조합에서 반드시 깨집니다.

그래서 개별 화면이 스스로 켜지고 꺼지게 두는 대신, 관리 주체를 하나 두고 그쪽에서 무엇을 띄울지 결정하는 구조로 갔습니다. 대화 중에는 이동 입력을 막고, 팝업이 떠 있는 동안에는 그 아래 화면이 입력을 받지 않게 하는 규칙을 한 곳에 모아 두는 편이, 화면이 늘어날 때마다 조건문이 여기저기 번지는 것보다 낫다고 판단했습니다.

4. 전투 — 무기 콜라이더로 타격 판정

플레이어의 공격 · 피격 · 사망과 스킬 슬롯 4종이 동작하도록 구현했습니다. 무기는 지팡이와 창 두 종류, 적은 숲 구간에 배치된 정령형 몬스터입니다.

명중 판정은 무기에 붙인 콜라이더를 공격 모션 중에만 켜는 방식으로 처리했습니다. 공격 버튼을 누르면 모션이 재생되고, 실제로 타격이 일어나는 구간에서만 콜라이더가 열려 그 사이에 닿은 적에게 데미지가 들어갑니다.

이 방식을 고른 이유는 타격 타이밍을 눈에 보이는 모션과 일치시킬 수 있기 때문입니다. 거리 계산으로 판정하면 버튼을 누른 순간 이미 맞은 것이 되어, 무기가 아직 뒤에 있는데 적이 날아가는 그림이 나옵니다. 콜라이더를 모션에 맞춰 여닫으면 "보이는 대로 맞는" 감각이 유지되고, 타격감이 어색할 때 판정 구간만 앞뒤로 옮겨 조절할 수 있습니다.

사망하면 게임오버 화면으로 넘어가 다시하기로 복귀할 수 있습니다.

5. 몬스터 AI — 엔진에 뼈대를 맡기고 판단만 직접

숲 구간의 몬스터가 플레이어를 인식해 추적하고 공격하도록 구현했습니다. 마을은 안전 구역, 숲은 전투 구역으로 나뉘어 플레이어가 어디서 긴장해야 하는지가 공간으로 구분됩니다.

구성은 세 겹입니다.

담당	무엇을
NavMesh (Unity AI Navigation)	어디로 어떻게 갈지 — 나무와 지형을 피해 플레이어에게 도달하는 경로
Animator 상태 머신	지금 무슨 상태인지 — 대기 · 추적 · 공격 · 사망과 그에 맞는 애니메이션
커스텀 스크립트	언제 상태를 바꿀지 — 플레이어를 인식했는가, 사거리에 들어왔는가

길찾기를 직접 짜지 않은 것은 의도적입니다. 엘브네아의 숲은 나무와 기복이 있는 3D 공간이라 경로 탐색이 필요한데, 이걸 직접 구현하면 그 자체로 프로젝트 하나 분량이 되고
그렇게 만들어도 NavMesh 보다 나은 결과가 나올 이유가 없었습니다.

상태를 Animator 로 관리한 것도 같은 판단입니다. 상태를 스크립트에서 따로 들고 있으면 "코드는 공격 상태인데 화면은 아직 걷는 모션" 같은 어긋남이 생기고, 이걸 맞추는 코드가 계속 늘어납니다. 상태와 애니메이션을 한 곳에서 같이 쥐고 있으면 그 어긋남이 애초에 발생하지 않습니다. 4장의 무기 콜라이더도 결국 모션 위에서 여닫히므로, 전투와 AI가 같은 축에서 관리된다는 이점도 있었습니다.

6. 대화 · 퀘스트 시스템 — 에셋을 뜯어 쓰다

Pixel Crushers Dialogue System 을 개조해 NPC 대화와 메인 퀘스트 진행을 연결했습니다.

직접 만들지 않은 이유

첫째, 당시 제 C# 숙련도로 대화 분기와 진행 상태 저장까지 직접 짜는 것은 기간 안에 끝낼 일이 아니었습니다.
둘째, 3인 팀에서 구현 담당이 한 명이면 만들 수 있는 총량이 정해져 있습니다. 대화에 시간을 쓰면 전투와 AI가 밉니다.

그대로 쓰지 않고 고친 곳

에셋을 붙이기만 한 것이 아니라, 이 게임에 맞게 세 군데를 손봤습니다.

1. 대화창 UI 교체 — 기본 스킨 대신 엘브네아의 UI 원화(녹색 테두리 패널)에 맞춰 다시 만들었습니다. 아트가 잡아둔 톤과 대화창만 따로 노는 것을 막기 위해서입니다.
2. 퀘스트 진행 연동 — 대화 결과가 퀘스트 상태를 바꾸고, 그 상태가 다시 상단 배너(Quest No.1 : 주변을 둘러보자!)와 우측 목표 리스트에 반영되도록 연결했습니다.
3. 다른 시스템 호출 — 대화 도중 아이템 획득 팝업을 띄우고 사운드를 재생하도록, 3장의 UI 관리 시스템과 오디오 쪽을 대화 흐름에서 부를 수 있게 했습니다.

세 번째가 이 작업의 핵심입니다. 대화 시스템이 단순히 글자를 띄우는 창에서 끝나지 않고, 퀘스트 · UI · 사운드를 함께 움직이는 진행의 축이 되었습니다. "에셋을 붙였다"와 "에셋을 뜯어 게임 구조에 맞췄다"는 여기서 갈립니다.

7. 연출과 최적화

오디오와 이펙트를 배치하고 파티클 시스템을 최적화했습니다. 연출을 살리는 것과 프레임을 지키는 것은 서로 반대 방향으로 당기는 요구라, 어느 선까지 걸어낼지를 정하는 것이 실제 작업의 대부분이었습니다.

8. 로컬라이제이션

유니티짱(Unity-chan) 보이스를 적용했습니다. 유니티짱 라이선스는 크레딧 표기가 필수 조건이며, 이를 확인해 빌드에 표기를 넣었습니다.

외부 리소스를 쓸 때 라이선스 조건을 확인하고 반영하는 것까지가 사용의 일부라고 봅니다.

9. 트러블슈팅 — 졸업전시 현장

무슨 일이 있었나 전시 반응은 "아기자기하고 귀엽다"는 평이 가장 많았습니다. 그런데 개발 중 발견하지 못한 버그가 현장에서 다수 드러나, 관람객이 진행하지 못하고 멈추는 경우가 있었습니다.

어떻게 대응했나 현장에서 나온 버그는 그 자리에서 재현 조건을 찾아 수정하며 대응했습니다.

근본 원인 출품 전 점검 절차가 없었다는 것입니다. 개발자가 아는 경로로만 확인했기 때문에, 처음 잡아보는 사람이 하는 조작에서 문제가 터졌습니다. 만든 사람은 "이렇게 하면 되는 길"을 알고 있어서, 그 길로만 몇 번이고 확인합니다. 처음 잡는 사람은 그 길을 모르니 아무 데나 누릅니다. 버그는 전부 후자에서 나왔습니다.

이후 바뀐 것 빌드가 나오면 시연 시나리오를 따라 처음부터 끝까지 훑는 QA 절차를 먼저 잡습니다. "내가 아는 경로"가 아니라 "처음 보는 사람의 경로"로 확인해야 한다는 것이 이 프로젝트의 결론입니다.

10. 회고

3인 팀에서 구현을 혼자 맡으면 무엇을 직접 만들지 고르는 일이 곧 설계가 됩니다. 길찾기를 NavMesh 에, 상태를 Animator 에, 대화를 Pixel Crushers 에 맡긴 것은 "그게 더 좋은 기술이라서"가 아니라 남은 시간 안에 완성된 게임을 내놓기 위한 선택이었습니다. 대신 그렇게 아낀 시간은 화면 흐름을 한 곳에서 관리하는 일과, 대화 시스템을 뜯어 퀘스트 · UI · 사운드에 연결하는 일에 넣었습니다. 엔진이 대신 해주지 않는 부분이기 때문입니다.

가장 크게 배운 것은 마지막에 왔습니다. 게임은 돌아갔지만 처음 잡는 사람 앞에서는 돌아가지 않았습니다. 만드는 것과 남에게 보여줄 수 있는 상태로 만드는 것은 다른 일이고, 그 사이를 메우는 것이 QA라는 걸 전시장에서 배웠습니다.
